

卡尔曼滤波器结合了递归算法和数据融合的思想,下面通过具体的例子来直观地说明这两个概念。如图 6.1.1 所示,一台无人机使用压力传感器(气压计)来测量高度。 $z_{[k]}$ 表示测量的结果,下标 k 代表第 k 次测量。在实际情况中,由于测量过程中存在随机误差,每次测量的结果都会有所不同。

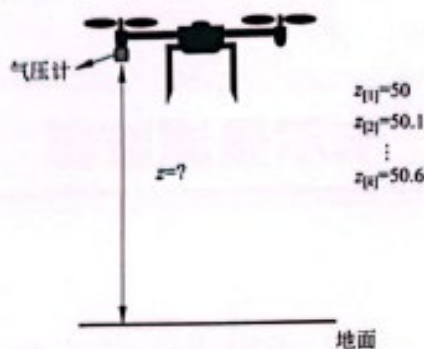


图 6.1.1 测量举例

为了消除测量误差的影响,我们可以通过多次测量无人机高度,并对这些测量值求平均值来获得一个更可靠的结果。在通过多次测量取平均值的过程中,由于每次测量的误差可能是随机的,并且正负偏差可能互相抵消,平均值将更接近真实值。用多次测量的平均值作为估计结果,即

$$\hat{z}_{[k]} = \frac{1}{k}(z_{[1]} + z_{[2]} + \cdots + z_{[k]}) \quad (6.1.1)$$

其中, $\hat{z}_{[k]}$ 代表经过 k 次测量后的估计值,“ $\hat{\cdot}$ ”代表估计,它是前 k 次测量的平均数。式(6.1.1)是我们在日常生活中使用最多的求平均值的方法。它存在两个缺陷:第一,每一次运算都需要做 k 次加法和 1 次乘法,随着 k 的增加,加法计算量会逐步增大;第二,每一次的测量结果都需要保留, k 次测量就需要存储 k 组结果,占用存储空间。因此,可以对式(6.1.1)稍加改进,得到

$$\begin{aligned} \hat{z}_{[k]} &= \frac{1}{k}(z_{[1]} + z_{[2]} + \cdots + z_{[k]}) \\ &= \frac{1}{k}(z_{[1]} + z_{[2]} + \cdots + z_{[k-1]}) + \frac{1}{k}z_{[k]} \\ &= \frac{k-1}{k} \left[\frac{1}{k-1}(z_{[1]} + z_{[2]} + \cdots + z_{[k-1]}) \right] + \frac{1}{k}z_{[k]} \end{aligned} \quad (6.1.2a)$$

其中, $\frac{1}{k-1}(z_{[1]} + z_{[2]} + \cdots + z_{[k-1]}) = \hat{z}_{[k-1]}$ 是 $k-1$ 次测量后的平均值,代入式(6.1.2a)可得

$$\begin{aligned} \hat{z}_{[k]} &= \frac{k-1}{k}\hat{z}_{[k-1]} + \frac{1}{k}z_{[k]} \\ &= \hat{z}_{[k-1]} - \frac{1}{k}\hat{z}_{[k-1]} + \frac{1}{k}z_{[k]} \\ &= \hat{z}_{[k-1]} + \frac{1}{k}(z_{[k]} - \hat{z}_{[k-1]}) \end{aligned} \quad (6.1.2b)$$

式(6.1.2b)是多次测量后平均值的另一种表达形式。

对比式(6.1.1),式(6.1.2b)只需要进行两次加法运算和一次乘法运算,可以快速地进行实时运算。同时,在每一次计算后,它只需要保留上一次的估计值 $\hat{z}_{[k-1]}$ 和当前次数 k ,这将大大减少硬件存储空间的使用。这种通过上一次估计值推断当前估计值的算法被称为**递归算法(recursive algorithm)**。

观察式(6.1.2b),当测量次数 k 很大时, $\hat{z}_{[k]} \approx \hat{z}_{[k-1]}$,新的测量结果对估计值的影响就变得很小。这是因为针对“重复测量取平均值”这一算法,多次测量已经积累了足够的信心,后续的测量就不再重要。反之,在最初的几次测量中(当 k 较小时),测量值与上一次估计值之间的差就会对新的估计值产生较大的影响。定义式(6.1.2b)中的 $\frac{1}{k} \triangleq K_{[k]}$,式(6.1.2b)可以写成

$$\hat{z}_{[k]} = \hat{z}_{[k-1]} + K_{[k]}(z_{[k]} - \hat{z}_{[k-1]}) \quad (6.1.3)$$

此时的 $K_{[k]}$ 是一个调整系数,在本例中 $K_{[k]} \in [0,1]$,且随着 k 的增加而减小。式(6.1.3)体现了数据融合(data fusion)的思想,通过调整系数将测量值与上一次的估计值融合在一起。

传感器融合

6.2.3 测量误差融合案例

在工程中,测量误差会被考虑为服从正态分布。基于这一思想,可以将多个带有误差的测量数据融合在一起,得到一个相对准确的测量值。考虑上述无人机高度测量的例子,如果我们在气压计之外增加一个超声波传感器,如图 6.2.2 所示,假如通过气压计测得的高度 $z_1 = 50\text{m}$,通过超声波传感器测得的高度 $z_2 = 52\text{m}$,下面将讨论如何科学地将两者融合在一起并得到一个理论上最优的估计值。

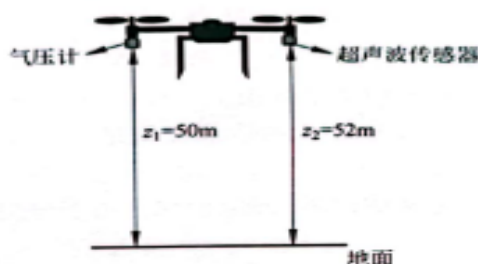


图 6.2.2 数据融合案例

设想这两种传感器的测量精度完全一样,那么可以简单地取它们的平均值作为估计值,即 $\hat{z} = \frac{1}{2}(z_1 + z_2) = 51\text{m}$ 。而在现实生活中,每一个传感器都有自己的精度范围,测量时产生的随机误差也不相同。定义这两个传感器的误差分别为

$$e_1 = z_a - z_1 \quad (6.2.8a)$$

$$e_2 = z_a - z_2 \quad (6.2.8b)$$

其中, z_a 是无人机在 k 时刻的真实高度值(下标 a 代表 actual)。在本例中,我们假设气压计的测量误差服从正态分布 $e_1 \sim N(0, \sigma_{e_1}^2)$,其中期望 $E(e_1) = 0$,标准差 $\sigma_{e_1} = 1\text{m}$,当使用气压计的测量结果 $z_1 = 50\text{m}$ 时,其真实结果的概率分布如图 6.2.3(a)所示,真实值有 99.73% 的可能性会落在 ± 3 个误差的标准差之内,即 47m 与 53m 之间。同理,假设超声波

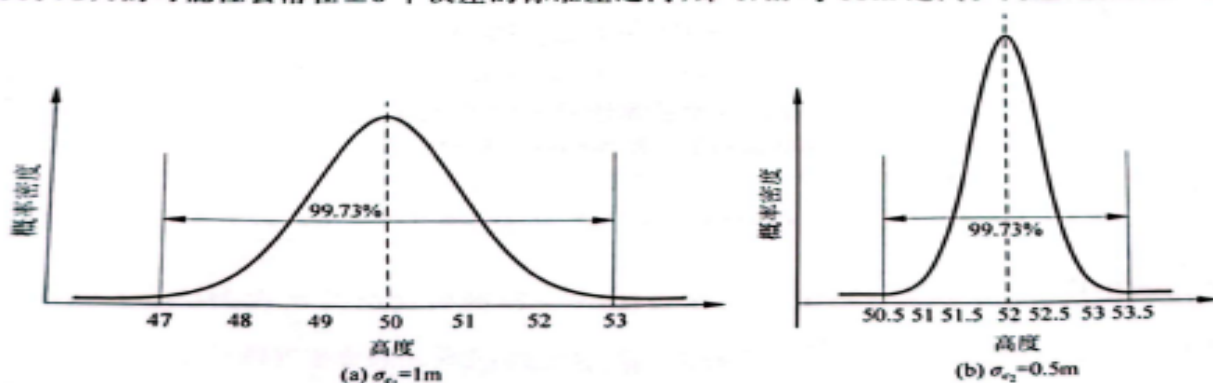


图 6.2.3 气压计与超声波传感器的测量结果概率分布

传感器的测量误差服从正态分布 $e_2 \sim N(0, \sigma_{e_2}^2)$, 其中期望 $E(e_2) = 0$, 标准差 $\sigma_{e_2} = 0.5 \text{ m}$, 通过它测量的真实结果概率分布如图 6.2.3(b) 所示。由于超声波传感器测量误差的标准差比较小 ($\sigma_{e_1} < \sigma_{e_2}$), 所以它在图中所形成的钟形曲线比气压计的曲线更加“修长”, 真实值更大概率地集中在测量值 $z_2 = 52 \text{ m}$ 周围。

下一步就是将这两条曲线通过某种方式融合, 形成一个误差标准差更小(更加细长的钟形曲线)的估计值。参考式(6.1.3), 一个通用的数据融合算法可以写成

$$\hat{z} = z_1 + K(z_2 - z_1) \quad (6.2.9)$$

其中, $K \in [0, 1]$ 。在式(6.2.9)中, 如果测量结果 z_1 的精度远远高于 z_2 (误差小, 即 $\sigma_{e_1} \ll \sigma_{e_2}$), 可以令参数 $K = 0$, 此时的估计值 $\hat{z} = z_1$ 。相反, 如果测量结果 z_1 的精度远远低于 z_2 (即 $\sigma_{e_1} \gg \sigma_{e_2}$), 可令参数 $K = 1$, 此时的估计值 $\hat{z} = z_2$ 。当 $K = 0.5$ 时, 则取两者的平均值。在本例中, $(\sigma_{e_1} = 1) > (\sigma_{e_2} = 0.5)$, 因此 K 的取值应该是一个大于 0.5 的数值, 这会使得计算结果偏向于 z_2 。

为了求解最优的参数 K , 定义估计误差为

$$\hat{e} = z_a - \hat{z} \quad (6.2.10a)$$

将式(6.2.9)代入式(6.2.10a), 可得

$$\begin{aligned} \hat{e} &= z_a - (z_1 + K(z_2 - z_1)) \\ &= z_a - z_1 - K(z_2 - z_1) \\ &= (z_a - z_1) + K(z_1 - z_2 + z_a - z_a) \\ &= (z_a - z_1) + K((z_a - z_2) - (z_a - z_1)) \end{aligned} \quad (6.2.10b)$$

将式(6.2.8)代入式(6.2.10b), 可得

$$\hat{e} = e_1 + K(e_2 - e_1) \quad (6.2.11)$$

$\hat{e}$ 是 e_1 与 e_2 的线性组合且 $E(e_1) = E(e_2) = 0$, 其期望为

$$\begin{aligned} E(\hat{e}) &= E(e_1 + K(e_2 - e_1)) \\ &= E((1 - K)e_1 + Ke_2) \\ &= (1 - K)E(e_1) + KE(e_2) = 0 \end{aligned} \quad (6.2.12a)$$

估计误差的方差为

$$\begin{aligned} \sigma_{\hat{e}}^2 &= \text{Var}(e_1 + K(e_2 - e_1)) \\ &= \text{Var}((1 - K)e_1 + Ke_2) \end{aligned} \quad (6.2.12b)$$

由于这两次测量是相互独立的, 根据方差运算性质式(6.2.6d), 式(6.2.12a)可以写成

$$\sigma_{\hat{e}}^2 = \text{Var}((1 - K)e_1) + \text{Var}(Ke_2) \quad (6.2.12c)$$

根据式(6.2.6b), 可得

$$\sigma_{\hat{e}}^2 = (1 - K)^2 \sigma_{e_1}^2 + K^2 \sigma_{e_2}^2 \quad (6.2.12d)$$

令 $\frac{d\sigma_{\hat{e}}^2}{dK} = 0$, 即

$$\frac{d\sigma_{\hat{e}}^2}{dK} = -2(1 - K)\sigma_{e_1}^2 + 2K\sigma_{e_2}^2 = 0$$

$$\Rightarrow K^* = \frac{\sigma_{e_1}^2}{\sigma_{e_1}^2 + \sigma_{e_2}^2} \quad (6.2.13a)$$

将 $\sigma_{e_1}=1$ 和 $\sigma_{e_2}=0.5$ 代入式(6.2.13a), 可得

$$K^* = \frac{\sigma_{e_1}^2}{\sigma_{e_1}^2 + \sigma_{e_2}^2} = \frac{1}{1^2 + 0.5^2} = 0.8 \quad (6.2.13b)$$

σ_e^2 对 K 的二阶导数为

$$\frac{d^2 \sigma_e^2}{dK^2} = \frac{d(-2(1-K)\sigma_{e_1}^2 + 2K\sigma_{e_2}^2)}{dK} = 2(\sigma_{e_1}^2 + \sigma_{e_2}^2) > 0 \quad (6.2.13c)$$

因此当 $K^* = \frac{\sigma_{e_1}^2}{\sigma_{e_1}^2 + \sigma_{e_2}^2} = 0.8$ 时, σ_e^2 为最小值。代入式(6.2.9), 可以得到最优估计值

$$\hat{z} = z_1 + K^*(z_2 - z_1) = 50 + 0.8 \times (52 - 50) = 51.6 \quad (6.2.14a)$$

代入式(6.2.12d), 得到最优估计误差的标准差

$$\sigma_e = \sqrt{(1-K^*)^2 \sigma_{e_1}^2 + (K^*)^2 \sigma_{e_2}^2} = 0.45 \quad (6.2.14b)$$

这一标准差小于每一个传感器单独测量的误差标准差($\sigma_e < \sigma_{e_2} < \sigma_{e_1}$)。数据融合结果如图 6.2.4 所示。

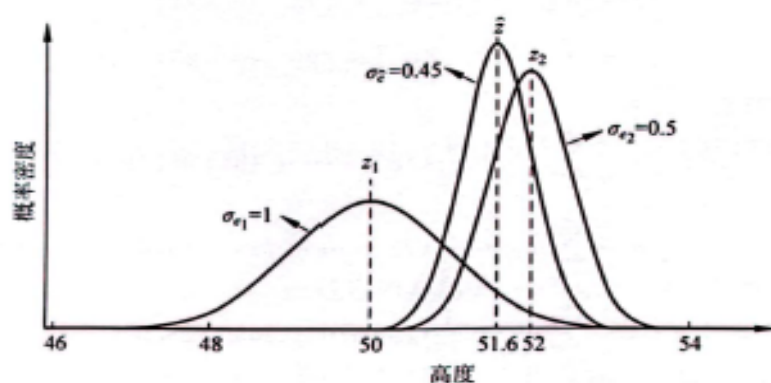


图 6.2.4 数据融合结果

MPC

想象你正在淋浴，希望水温保持在舒适的 40°C 。你面前有一个水龙头：向左转是热水，向右转是冷水。但水管很长，拧动龙头后要过几秒水温才会变化。这就有了**延迟**和**惯性**。你该如何调节？

这就是MPC要解决的问题：**用模型预估未来，提前做决定，边做边修正。**

下面我们用这个“淋浴水温控制”例子，一步步推导MPC的原理和计算过程。

1. 建立最简单的“模型”

首先，我们要用一个数学式子描述“水龙头转动角度如何影响未来水温”。假设每1秒决策一次，离散时间 $k = 0, 1, 2, \dots$ 。

记

- $T(k)$: 第 k 秒时的实测水温 ($^{\circ}\text{C}$)
- $u(k)$: 第 k 秒时你转动龙头的角度 (正值表示加热，负值表示降温)

一个非常简化的模型（忽略延迟）是：

$$T(k+1) = T(k) + 0.5 \cdot u(k)$$

解释：当前水温加上“控制作用”的一半，就是下一秒的水温。比如现在 40°C ，你给 $u = 2$ （转热一点），那么下一秒水温会上升 $0.5 \times 2 = 1^{\circ}\text{C}$ ，变成 41°C 。

但实际淋浴有**延迟**：拧龙头后要3秒才能看到水温变化。所以我们引入更真实的模型（含延迟1秒）：

$$T(k+1) = 0.9 T(k) + 0.4 u(k-1)$$

其中 $u(k-1)$ 是你上一秒的龙头角度。这表示：你这一秒的动作，要到下一秒才影响温度。而且温度还会自然散热（系数0.9）。为了简化，我们先用无延迟模型演示核心思想，最后说明如何处理延迟。

2. MPC的核心思想：预测 + 滚动优化

步骤概要：

1. **预测**：用当前温度 $T(k)$ 和未来的控制动作 $u(k), u(k+1), \dots$ 预测未来 N 步的温度（ N 称为**预测时域**）。
2. **优化**：找到一个控制序列（例如未来3步的动作），使得预测温度尽可能接近目标（ 40°C ），同时动作不要太大、太剧烈。
3. **执行**：只执行这个序列的第一步 $u(k)$ ，然后等待1秒，测得新的真实温度 $T(k+1)$ 。
4. **反馈**：用新测量值重新预测，重复以上过程。

这就像你淋浴时：先预估未来3秒的温度变化，找出“接下来3秒最优的拧龙头方式”，然后只做第1秒的动作，1秒后再重新评估。

3. 具体数值推导

假设当前 $k = 0$ 时刻，实测水温 $T(0) = 20^{\circ}\text{C}$ （冷水），目标 $T_{ref} = 40^{\circ}\text{C}$ 。

模型（无延迟简化版）：

$$T(k+1) = T(k) + 0.5 u(k)$$

选择预测时域 $N = 3$ （预测未来3步：第1,2,3秒的温度），控制时域 $M = 2$ （只优化前2秒的动作，第3秒后动作保持为0）。

3.1 写出预测方程

令当前时刻为 $k = 0$ ，已知 $T(0) = 20$ 。

未来温度：

$$T(1) = T(0) + 0.5u(0) = 20 + 0.5u(0)$$

$$T(2) = T(1) + 0.5u(1) = 20 + 0.5u(0) + 0.5u(1)$$

$$T(3) = T(2) + 0.5u(2) = 20 + 0.5u(0) + 0.5u(1) + 0.5u(2)$$

由于控制时域 $M = 2$ ，我们设 $u(2) = 0$ （第3秒不再主动调节）。所以：

$$T(3) = 20 + 0.5u(0) + 0.5u(1)$$

3.2 定义代价函数 (优化目标)

我们希望温度尽快到40°C, 且动作不要太剧烈。常用二次型代价:

$$J = \sum_{i=1}^3 [T(i) - 40]^2 + \lambda \sum_{j=0}^1 [u(j)]^2$$

第一项是温度偏差惩罚, 第二项是控制动作惩罚, λ 是权重 (这里取 $\lambda = 0.1$ 表示稍微在意动作幅度)。

代入预测表达式:

$$J = [20 + 0.5u_0 - 40]^2 + [20 + 0.5u_0 + 0.5u_1 - 40]^2 + [20 + 0.5u_0 + 0.5u_1 - 40]^2 + 0.1(u_0^2 + u_1^2)$$

注意 $T(2)$ 和 $T(3)$ 表达式相同 (因为 $u_2 = 0$) , 所以第二项和第三项一样。简化:

$$\text{令 } e_1 = 20 + 0.5u_0 - 40 = -20 + 0.5u_0$$

$$e_2 = 20 + 0.5u_0 + 0.5u_1 - 40 = -20 + 0.5u_0 + 0.5u_1$$

则

$$J = e_1^2 + 2e_2^2 + 0.1(u_0^2 + u_1^2)$$

3.3 求解最优 u_0, u_1

为了找到使 J 最小的 u_0, u_1 , 分别对它们求偏导并令为0。

先展开:

$$e_1 = -20 + 0.5u_0$$

$$e_2 = -20 + 0.5u_0 + 0.5u_1$$

$$J = (-20 + 0.5u_0)^2 + 2(-20 + 0.5u_0 + 0.5u_1)^2 + 0.1u_0^2 + 0.1u_1^2$$

对 u_0 求导:

$$\frac{\partial J}{\partial u_0} = 2(-20 + 0.5u_0) \cdot 0.5 + 4(-20 + 0.5u_0 + 0.5u_1) \cdot 0.5 + 0.2u_0 = 0$$

计算:

$$\text{第一项: } 2 \cdot 0.5 = 1 \rightarrow (-20 + 0.5u_0)$$

$$\text{第二项: } 4 \cdot 0.5 = 2 \rightarrow 2(-20 + 0.5u_0 + 0.5u_1)$$

所以:

$$(-20 + 0.5u_0) + 2(-20 + 0.5u_0 + 0.5u_1) + 0.2u_0 = 0$$

$$-20 + 0.5u_0 - 40 + u_0 + u_1 + 0.2u_0 = 0$$

合并常数: -60 ; 合并 u_0 : $0.5 + 1 + 0.2 = 1.7u_0$; 再加上 u_1 :

$$-60 + 1.7u_0 + u_1 = 0 \Rightarrow 1.7u_0 + u_1 = 60 \quad (1)$$

对 u_1 求导:

$$\frac{\partial J}{\partial u_1} = 4(-20 + 0.5u_0 + 0.5u_1) \cdot 0.5 + 0.2u_1 = 0$$

即:

$$\begin{aligned} 2(-20 + 0.5u_0 + 0.5u_1) + 0.2u_1 &= 0 \\ -40 + u_0 + u_1 + 0.2u_1 &= 0 \Rightarrow -40 + u_0 + 1.2u_1 = 0 \quad (2) \end{aligned}$$

解方程组:

由(2): $u_0 = 40 - 1.2u_1$

代入(1): $1.7(40 - 1.2u_1) + u_1 = 60 \Rightarrow 68 - 2.04u_1 + u_1 = 60 \Rightarrow 68 - 1.04u_1 = 60$

得 $1.04u_1 = 8 \Rightarrow u_1 \approx 7.692$

然后 $u_0 = 40 - 1.2 \times 7.692 \approx 40 - 9.2304 = 30.7696$

所以最优控制序列: 第一步 $u_0 \approx 30.8$, 第二步 $u_1 \approx 7.7$ 。

3.4 执行第一步

MPC只执行 $u_0 = 30.8$ 。假设你转动水龙头到这个角度(很大, 因为初始水温很低)。1秒后, 实际水温变为:

$$T(1) = 20 + 0.5 \times 30.8 = 20 + 15.4 = 35.4^\circ\text{C}$$

3.5 反馈与滚动

现在 $k = 1$, 实测 $T(1) = 35.4$ (可能因为模型不准或外界扰动, 不一定是35.4? 但这里按模型计算正好是35.4), 目标仍是40。重新预测未来3步 ($T(2), T(3), T(4)$), 重新优化求新的 u_1 (注意这里的 u_1 是新的第一步)。重复这个过程。

你会发现, 随着温度接近目标, 计算出的控制量会逐渐减小, 最后稳定在某个值使水温维持 40°C 。